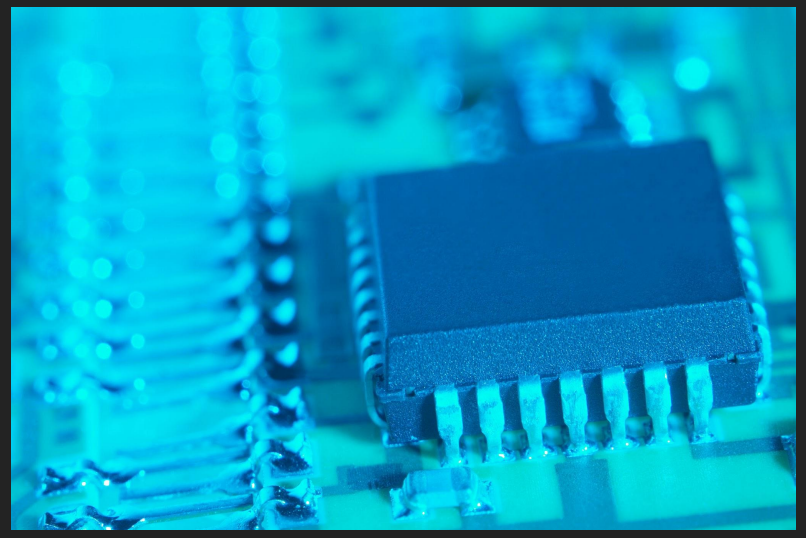
TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 5:Embedded Systems and Robot Hardware Interfaces

TURTLE Incubator: SO-ARM Software Stack





Week 5: Agenda

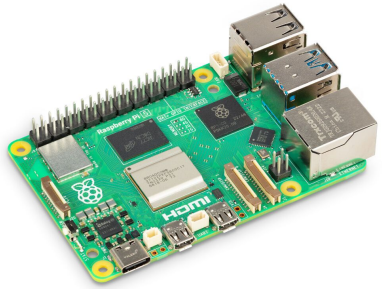
- Embedded Systems Overview
- Communication Protocols
- Actuators
- Sensors
- Reading Datasheets
- SO-ARM Application
- Demonstration
- Hands-On: Train the SO-ARM
- Deliverable → Record a dataset and start training a policy

Embedded Systems Overview

- **MCU vs SBC**
  - Microcontroller (MCU): An integrated circuit device used to control a system of electronics via a microprocessor, onboard memory, and various peripherals.
  - Single Board Computer (SBC): Has all the features of a complete computer within one single board.



Microcontrollers control devices by inputting and outputting signals to various parts of an electronic device. They generally run on 1 singular program and are dedicated to 1 task



Single Board Computers can perform many different tasks and can have a variety of features like signal processing, wireless communication, artificial intelligence, or even some computer operating systems.

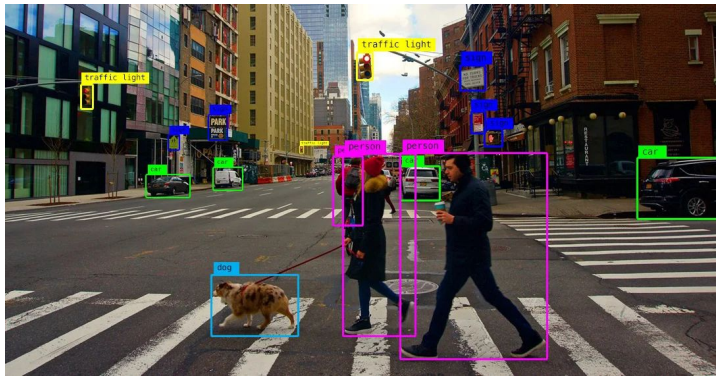
https://www.seeedstudio.com/blog/2020/10/27/all-about-cpus-microprocessor-microcontroller-and-single-board-computer/?srsltid=AfmBOoo00CkH7xEmf7TcUtJz5fl0pn55xy9pCN0WfkEqmmCgie6H3xQy

Embedded Systems Overview

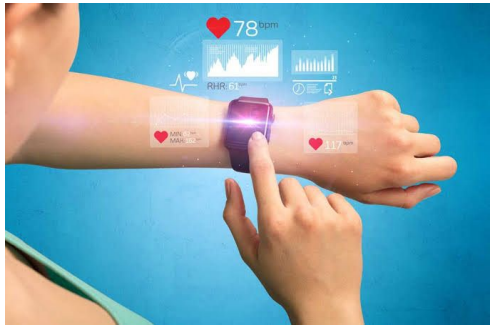
- **Real-Time Systems**
  - Real-Time Systems are information processing systems that perform functions within predictable and specific time constraints (in real time)
  - To qualify as a real time system, a device needs to be able to complete its tasks by a specified deadline and be able to coordinate its clock with other devices so they can operate in unison
  - Some Examples below:



Multiplayer Video Games



Computer Vision



Wearable Technology

In the above examples of Real-Time Systems, the device needs to take in data, then analyze and apply functions on it in order to provide a seamless user experience

https://www.intel.com/content/www/us/en/learn/what-is-a-real-time-system.html

Embedded Systems Overview

- **Robotics Hardware Architecture**

https://www.cs.cmu.edu/~reids/papers/Robot_Architectures.pdf

Communication Protocols

- There are many common ways for devices to communicate with each other, some common communication protocols are described and their features are shown below.

- **UART:** One of the earliest communication protocols, consists of 1 device communicating to another device.
- **I2C:** Versatile protocol capable of interfacing 1 master controller with many low speed devices.
- **SPI:** Capable of transmitting data very quickly to many devices but has short range and requires a new write for every slave device.
- **CAN:** CAN nodes use message ID to communicate without a singular master, this makes it robust and fast.

	UART	I2C	SPI	CAN
Wires	2 (TX, RX)	2 (SDA, SCL)	4 (SCK, MOSI, MISO, CS)	2 (CAN-H, CAN-L)
Topology	P2P	1 Master, Many Slaves	1 Master, Many Slaves	Multiple Masters Message ID
Speeds	9600 - 115200 bps	100 kbit/s up to 5 Mbit/s	10-20+ Mbit/s	1 Mbit/s at 40m
Range	Short	~1m	~10cm	up to 1km
Error Handling	Optional parity bit	ACK/NACK bits	none	CRC, ACK slot, Automatic retransmission
Uses	Modem,GPS, debug console	multiple low speed sensors sharing wires	Fast peripherals, SD cards, displays	Vehicles, industrial equipment
Extra	both sides need to agree on baud rate	Pull-up resistor sizing, address collisions	Wire count increases with each slave	needs an external transceiver IC and 120 ohm resistors

https://dev.blues.io/blog/blues-university-understanding-sensor-interfaces-uart-i2c-spi-can/

Actuators

- **Choosing the correct type of actuator for your project is very important. The SO-Arm will use servos Below are some different types of actuators and their characteristics.**

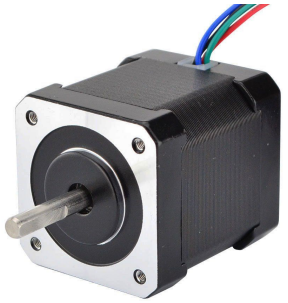
Servo

- 3 wires
- can be run directly off MCU
- High precision
- Built-in closed loop control maintains position



Stepper Motor

- 4 - 8 wires
- requires a stepper driver
- High precision
- usually only has open loop control, location can drift over time
- High torque at low speeds



DC Motor

- 2 Wires
- Cheap and simple setup
- requires a motor driver
- low efficiency



Brushless DC Motor

- 3 wires
- High efficiency
- High Speed
- Requires an ESC
- Can be expensive

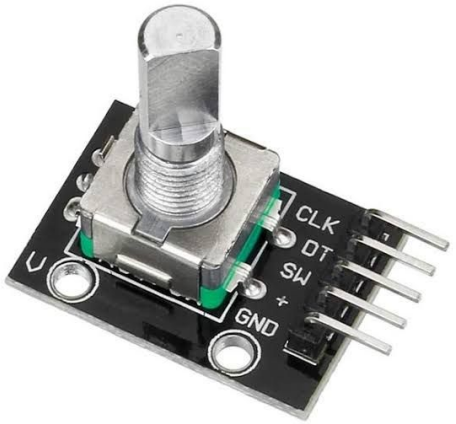


Sensors

Similarly to Actuators, choosing the correct sensor for your project is important for your robot to interact with the outside world precisely. The basic sensor types that the SO-Arm uses are shown below

Encoder

- Encoders are usually used to detect rotational movement/position



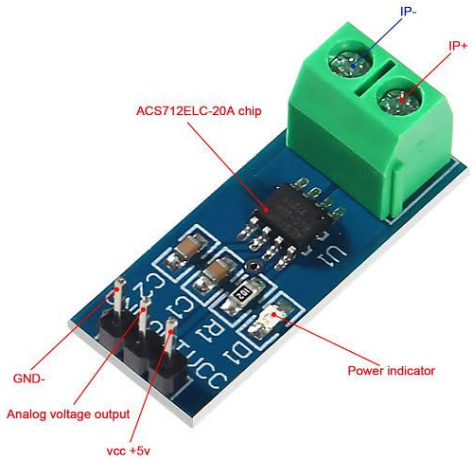
Position sensor

- A linear position sensor like this detects the position of the sensor along a shaft



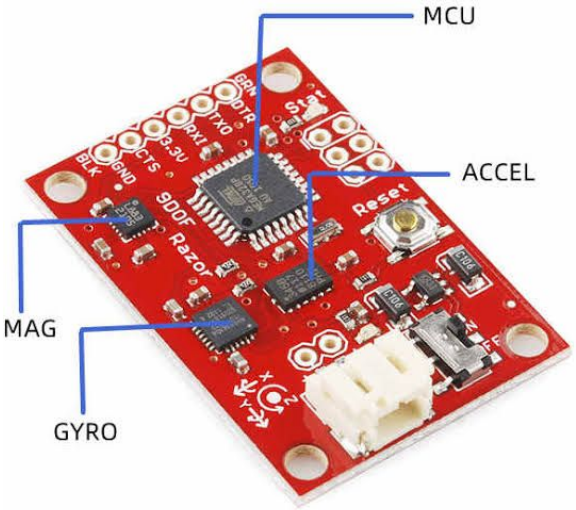
Current Sensor

- Current sensors detect current along a wire that it is in series with



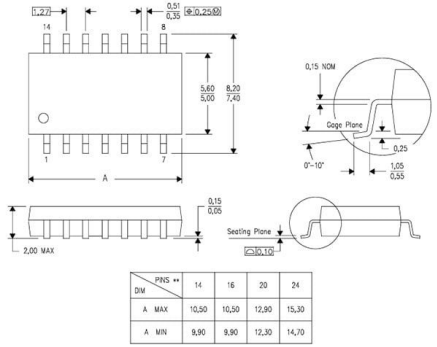
IMU

- Inertial Measurement Unit
- can usually detect acceleration, orientation and sometimes magnetic fields



Reading Datasheets

- **Important Specifications**



DIM \ PINS **	14	16	20	24
A MAX	10,50	10,50	12,90	15,30
A MIN	9,90	9,90	12,30	14,70

https://www.digikey.com/en/maker/tutorials/2024/how-to-read-and-understand-technical-datasheets

Reading Datasheets

- **Common Pitfalls**

https://www.digikey.com/en/maker/tutorials/2024/how-to-read-and-understand-technical-datasheets

SO-ARM Application

SO-ARM Application

- Sending commands to motors
- Reading encoder feedback
-
-

Demonstration

Demonstration: From Teleop to Training Data

Watch how moving the arm by hand becomes data a robot can learn from

1 Teleoperate

`lerobot-teleoperate`

Move the leader arm by hand. Its encoders are read and sent as position commands to the follower's servos, so the follower copies you in real time.

2 Record

`lerobot-record`

Add a camera and record a few short episodes. Every frame saves the camera image, the follower's joint positions, and the leader's commanded action.

3 Inspect

`LeRobot dataset visualizer`

Open the dataset on Hugging Face and scrub through an episode. Images, joint states and actions line up frame by frame. This is what the policy learns from.

Connect it to today: the servos (actuators) and encoders (sensors) we just covered are the robot's outputs and inputs. A trained policy learns to map camera image + joint positions to the next joint commands.

Hands-On

Hands-On: Train Your SO-ARM

Today you go from demonstrations to a training run

1 Set up

Arms, camera and Hugging Face login all working

2 Record

10+ teleop episodes of one simple task

3 Check

Review every episode in the dataset visualizer

4 Train

Start an ACT policy training run

What is ACT?

Action Chunking with Transformers

An imitation learning policy. It studies your demonstrations and learns to predict the next chunk of joint positions from the camera image and the arm's current joint positions. Better demos in means a better robot out.

Heads up on time

Training takes several hours. The goal today is a run that starts cleanly and shows the loss going down. Let it keep running after class.

Hands-On: Step 1, Set Up

1. **Activate your environment**
   Same conda env you used for calibration and teleop.

2. **Find your ports**
   Unplug and replug each arm with lerobot-find-port. Write down leader and follower COM ports.

3. **Mount a camera**
   USB webcam clamped or taped in place, looking down at the workspace. It must not move between episodes.

4. **Log in to Hugging Face**
   Create a token with Write access at huggingface.co/settings/tokens, then run hf auth login.

5. **Test teleop with the camera**
   Follower mirrors leader and the camera feed shows in the viewer. Fix this before recording.

Anaconda Prompt (yellow = change to yours)

```
conda activate lerobot
lerobot-find-port
lerobot-find-cameras opencv

hf auth login
hf auth whoami

# Teleop test (type the --robot.cameras value on ONE line)
lerobot-teleoperate ^
  --robot.type=so101_follower ^
  --robot.port=COM3 ^
  --robot.id=follower_arm ^
  --robot.cameras="{front: {type: opencv, index_or_path: 0,
       width: 640, height: 480, fps: 30}}" ^
  --teleop.type=so101_leader ^
  --teleop.port=COM7 ^
  --teleop.id=leader_arm ^
  --display_data=true
```

Hands-On: Step 2, Record Your Dataset

Anaconda Prompt

```
# Type the --robot.cameras value on ONE line
lerobot-record ^
  --robot.type=so101_follower ^
  --robot.port=COM3 ^
  --robot.id=follower_arm ^
  --robot.cameras="{front: {type: opencv, index_or_path: 0,
       width: 640, height: 480, fps: 30}}" ^
  --teleop.type=so101_leader ^
  --teleop.port=COM7 ^
  --teleop.id=leader_arm ^
  --display_data=true ^
  --dataset.repo_id=YOUR_HF_USER/so101_week5 ^
  --dataset.single_task="Pick up the cube and place it in the box" ^
  --dataset.num_episodes=10 ^
  --dataset.episode_time_s=20 ^
  --dataset.reset_time_s=10
```

Keyboard controls

→ or n	end episode early
← or r	redo this episode
Esc or q	stop and upload

Good demos

Do the task the same way every time

Keep the cube in the camera's view

Move smoothly, no long pauses

Vary the cube's start spot a little

10 episodes to start, about 50 for a reliable policy

Hands-On: Step 3, Check It and Start Training

Check first: open huggingface.co/spaces/lerobot/visualize_dataset, paste YOUR_HF_USER/so101_week5, and watch each episode. Redo any where the cube leaves the frame.

Anaconda Prompt

```
lerobot-train ^
  --dataset.repo_id=YOUR_HF_USER/so101_week5 ^
  --policy.type=act ^
  --output_dir=outputs/train/act_so101_week5 ^
  --job_name=act_so101_week5 ^
  --policy.device=cuda ^
  --policy.repo_id=YOUR_HF_USER/act_so101_week5

# Stopped early? Resume from the last checkpoint:
lerobot-train ^
  --config_path=outputs/train/act_so101_week5/
    checkpoints/last/pretrained_model/train_config.json ^
  --resume=true
```

Pick your device

NVIDIA GPU

`--policy.device=cuda`

Apple silicon Mac

`--policy.device=mps`

No GPU

Use the LeRobot ACT notebook on Google Colab (huggingface.co/docs/lerobot/notebooks)

Healthy run

Loss drops steadily in the log. Checkpoints save to outputs/train/.../checkpoints.

Hands-On: Common Mistakes

Arm asks to recalibrate

The --robot.id or --teleop.id doesn't match your calibration. Use the same IDs you calibrated with.

Leader and follower swapped

Ports got mixed up. Rerun lerobot-find-port and unplug each arm to confirm.

Upload fails at the end

Not logged in, or the token is read only. Run hf auth login with a Write token before recording.

Camera bumped mid-session

The policy sees a different scene than it trained on. Fix the camera in place and don't touch it.

"Dataset already exists"

That repo_id is already in your local cache. Pick a new name, or use the resume flag to add more episodes.

Training on a CPU

It would take days. Use cuda, mps, or the Colab notebook instead.

Deliverable: Start Training Your SO-ARM

Four checkpoints. Show each one to a mentor before you leave.

1

Teleop + camera

Follower mirrors the leader and the camera feed shows in the viewer.

2

Dataset recorded

At least 10 episodes of the cube task, uploaded to your Hugging Face account.

3

Dataset checked

Every episode reviewed in the visualizer, with bad ones re-recorded.

4

Training started

lerobot-train running locally or on Colab, with the loss going down.

Submit: your dataset link (huggingface.co/datasets/YOUR_HF_USER/so101_week5) and a screenshot of your training log.

Additional Learning

Additional Learning

- Include some links to check out or something here



“The quote thing from the Hatchling Slides”

Khanathan Vo

Software Incubator